Program and Branch Name:
BTech CSE-F

PBL Progress Report – Final

Field	Details
Course Name	Computer Architecture & Parallel Processing
Course Code	CCSE0304
Faculty Name	Dr. Amrita Bhatnagar
Assignment Type	Group Assignment
Student(s) Name	Utkarsh Raj Shukla, Vivek Kumar, Vishal Gupta, Yash Srivastava, Nishant Kumar Mahto
ERP ID (I Year) / Roll No. (II Year onwards)	2501330100398, 2501330100414, 2501330100411, 2501330100422, 0261DCS009
Team/Group No. *(Group/Capstone only)*	Group No. 91
Problem Statement / Project Title	Pharmacy Inventory Computing System
SDG(s)	SDG 3 – Good Health & Well-Being
Reporting Month	Month 3 – Final
Date of Submission	29/10/2026

Student Declaration

I/We certify that this assignment is my/our own work, with all sources appropriately acknowledged. I/We understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarised or inappropriately AI-generated.

Student Name & Signature(s): Utkarsh Raj Shukla, Vivek Kumar, Vishal Gupta, Yash Srivastava, Nishant Kumar Mahto
Date: 29/10/2026

2. Student Details / Team Details

S.No.	Student Name	ERP ID / Roll No.	Role / Contribution
1	Utkarsh Raj Shukla	2501330100398	PharmaCPU ISA, assembler, instruction cycle, integration, GitHub
2	Vivek Kumar	2501330100414	ALU design, IEEE 754 encoding, floating-point arithmetic
3	Vishal Gupta	2501330100411	Division, Booth's algorithm, logic operations, status register
4	Yash Srivastava	2501330100422	Pipelining, RISC vs CISC, Flynn's taxonomy, testing
5	Nishant Kumar Mahto	0261DCS009	Stock view, control-design view, report compilation

3. Executive Summary

Small pharmacies that keep stock by hand often miss low-stock and expiry warnings. Our group built the Pharmacy Inventory Computing System, a web application in which every stock calculation runs on hardware we simulated from the CAPP syllabus. An 8-bit ALU handles sales, prices are IEEE 754 words, and inventory scans run on our 16-bit PharmaCPU with a 5-stage pipeline. Units 1 to 3 are complete and pass 84,374 automated checks. [ADD: one sentence on the Unit 4–5 result]

4. Introduction

A pharmacy has to know, for every medicine, how many units are left, when they expire and when to reorder. These are small calculations, but inside a computer they pass through registers, an ALU and a control unit [1]. We picked this problem because it lets each CAPP unit appear as a working feature instead of a diagram. Our objectives were to manage a real medicine list, to run each calculation on a simulated machine that the user can step through, and to link every syllabus unit to at least one feature. The scope is a browser application with a 10-medicine sample dataset.

5. Problem Definition

Manual records in small pharmacies cause stock-counting errors, missed expiry dates and late reorders, and staff get no warning before a medicine runs out. Our objectives were (1) to keep one record per medicine with batch, units, pack size, reorder level, unit price, expiry date, supplier and prescription or cold-storage flags, (2) to flag low-stock and expiring medicines automatically, and (3) to carry out every calculation through a visible ALU, floating-point unit, CPU and pipeline.

The target users are pharmacists, pharmacy staff and managers, with suppliers as secondary stakeholders. The app runs only in the browser (HTML, CSS and JavaScript), keeps its data in local storage and uses sample data. We counted the work as successful when every simulated result matched the expected value and the CPU scans found the same medicines as the Stock view. The project supports SDG 3, Good Health and Well-Being, because fewer stock-outs and expired medicines protect patients.

6. Industry / Application Overview

Retail and hospital pharmacies already use inventory and billing software to track batches, expiry dates and reorder points [ADD: names of the existing systems the group reviewed in Month 1]. Pharmacies without such software keep manual records, which Report I identified as the main source of errors. Commercial tools also hide how each number is calculated. That suits a pharmacist, but it hides what a CAPP student needs to see, such as the carry flag that blocks an impossible sale, the exponent and fraction of a price, and the pipeline stalls behind a stock scan. Our project fills this gap. It manages stock like a normal inventory tool, and any value can be opened to show the register transfers, ALU flags and micro-operations that produced it [2].

7. Module 1: Processor Organisation, Registers and Addressing

Concept: Unit 1, covering functional units and their connections, register and memory transfer, general register organisation and addressing modes [1], [3].

Approach: We modelled the system as one processor, PharmaCPU, with a program counter (PC), instruction register (IR), memory address and buffer registers (MAR, MBR) and eight general registers R0–R7, connected to a 512-word memory.

Work completed: The medicine count N is stored at 0x100. Each medicine then takes three words (quantity, reorder level, status flags) from 0x101, and results are written from 0x1F0. Our programs use four addressing modes: immediate (LDI R3, 0), direct (LOAD R2, [0x100]), register (SUB R6, R4, R5) and base plus displacement (LDR R4, [R1+0]). Every transfer is written in register transfer language, for example MAR ← PC.

Output: The Lab › Instruction cycle view shows every register after each clock.

Findings: Base plus displacement suited our record layout. R1 points to one medicine, offsets 0 and 1 reach its quantity and reorder level, and ADDI R1, 3 moves to the next record, so the same loop works for any number of medicines.

8. Module 2: ALU, Multiplication, Division and Logic Operations

Concept: Unit 2 (first part), covering ALU design, Booth's algorithm [4], restoring and non-restoring division, and logic operations [1].

Approach: Our n-bit ALU (8 bits in the Stock view) uses a 3-bit select S2 S1 S0 for ADD, SUB, AND, OR, XOR, NOT, SHL and SHR, and sets C, Z, N and V. Subtraction is A + B′ + 1, so C = 1 means no borrow.

Work completed: Selling 30 of 240 tablets runs SUB 240, 30 and gives 210 with C = 1. Selling 40 of 36 gives C = 0, a borrow, so the sale is refused. Booth's algorithm multiplies signed quantities. Both division methods split loose tablets into packs, for example 157 ÷ 10 = 15 packs and 7 tablets. In each medicine's 8-bit status register, OR sets bits, and the Reorder and Expiry filters are AND masks 0000 0011 and 0000 1100.

Output: Tests compare Booth's products with a × b for operands from −127 to 127, and both divisions with the true quotient and remainder.

Findings: Non-restoring division skips the restore step in each cycle but needs one final correction when A ends negative.

9. Module 3: Floating-Point Arithmetic and IEEE 754

Concept: Unit 2 (second part), covering the IEEE 754 single and double formats [5] and floating-point addition, subtraction, multiplication and division with correct rounding [6].

Approach: Each unit price is stored as a 32-bit word, and stock value is price × units in single precision. Every operation is shown in five stages: align the exponents, operate on the significands, normalise, round and pack. Unpacking gives the sign, the biased exponent and a 24-bit significand that includes the hidden 1.

Work completed: ₹6.40 is stored as 0x40CCCCCD, with sign 0, biased exponent 129 (a scale of 2^2) and fraction 0x4CCCCD. The value of 36 units at ₹6.40 is 230.40000915 in float32, because 6.40 has no exact binary form. Rounding is round-to-nearest-even using guard, round and sticky bits. The Lab › Floating point view lets the user enter any two numbers and follow each stage.

Output: The test script generates 20,000 random operand pairs and runs all four operations on each, so 80,000 results are compared with the browser's own float32 arithmetic (Math.fround). All 80,000 match bit for bit. Edge cases also match. For example, a single-precision significand has 24 bits, so $2^{24} + 1$ cannot be stored and 16777216 + 1 rounds to 16777216.

Findings: Our first version rounded some results wrongly. Keeping the guard, round and sticky bits through every stage removed all the mismatches, which showed us why the standard defines these extra bits.

10. Module 4: Control Unit, Instruction Cycle and Pipelining

Concept: Unit 3, covering instruction types and formats, the instruction cycle and micro-operations, program control, RISC and CISC, pipelining [3], hardwired and microprogrammed control with horizontal and vertical microcode [1], and Flynn's classification [7].

Approach: We designed an instruction set for PharmaCPU with 16 instructions in four types (data transfer, arithmetic, logical and program control) and five 16-bit formats (R, I, M, X and B), with the opcode in IR(15–12). We also wrote an assembler that turns inventory programs into machine words.

Work completed: Each instruction runs one clock at a time. Fetch is T0 MAR ← PC, T1 MBR ← M[MAR] and PC ← PC + 1, and T2 IR ← MBR. T3 decodes, and execution starts at T4. Hardwired control is shown as decoder and timing terms such as D6·T4 for SUB, and the same steps are stored in control memory as 24-bit horizontal and 26-bit vertical microcode words. JN, JZ and JNZ build the loops that scan every record. A sale takes LOAD, LOAD, SUB and STORE on our RISC machine, against one memory-to-memory SUBM on a CISC machine. A 5-stage pipeline (IF, ID, EX, MEM, WB) shows RAW stalls, forwarding and branch flushes. For Flynn's classification, an expiry scan runs as SISD and as 4-lane SIMD.

Output: The low-stock scan found Amoxicillin, Metformin and ORS, the same three medicines as the Stock view, in 85 instructions and 535 clock cycles. SIMD needed 4× fewer steps than SISD.

Findings: Forwarding cut the pipelined scan from 147 to 137 cycles. The remaining stalls come from a load followed straight away by its use, and from taken branches, which flush two instructions.

11. Module 5: Parallel Architectures and Shared-Memory Systems

Concept: Units 4 and 5, covering the memory hierarchy with cache and virtual memory, snooping coherence protocols (VI, MSI, MESI and Dragon), directory protocols, memory consistency models, synchronisation with LL-SC and barriers, and interconnection networks [8].

Approach: In Report II we planned to treat the pharmacy as a shared-memory multiprocessor. Each billing counter is a processor with a private cache of medicine records, and the shared stock list is main memory. Two counters selling the same

medicine at the same moment then become the coherence and synchronisation problem from the syllabus, and the branches of one pharmacy become nodes on an interconnect. The plan was a cache simulator with direct and set-associative mapping, MSI, MESI and Dragon coherence for several counters, directory protocols, LL-SC and barrier synchronisation for concurrent sales, and interconnects for multi-branch stores.

Work completed: [ADD: what the group built for Units 4 and 5. At the time of writing, the repository and the app list Units 4 and 5 as “Next review”.]

Output: [ADD: results, for example hit and miss rates or bus transactions per protocol]

Findings: [ADD: main findings from the Unit 4–5 work]

12. Final Integrated Solution / Testing and Validation

The modules combine into one web application with three views. The Stock view lists the medicines and lets the user search, filter, sell and restock. Each sale or restock calls the ALU from Module 2, each price is an IEEE 754 word from Module 3, and the Reorder and Expiry filters are AND masks on the status register. The Lab view has nine topics: ALU, division, multiplication, floating point, instruction set, instruction cycle, control design, pipelining and Flynn’s classification. The About view maps each CAPP unit to the app. [ADD: how the Module 5 work connects to these views]

We tested at three levels. First, a Node.js script runs 84,374 checks on the ALU flags, Booth’s multiplication, both division methods, IEEE 754 encoding, floating-point arithmetic, the inventory programs, assembler round trips, pipeline timing and the control memory. All pass, and GitHub Actions runs the script before every deployment to GitHub Pages. Second, the floating-point unit matched the browser’s float32 arithmetic on all 80,000 random cases. Third, we ran three inventory programs on the sample data, counting 5 cycles per instruction for the non-pipelined machine.

The low-stock count took 85 instructions, 425 cycles non-pipelined, 147 cycles pipelined without forwarding (2.89×) and 137 with forwarding (3.10×). The total-units program took 55 instructions and 275, 97 (2.84×) and 87 (3.16×) cycles. The expiry alert scan took 71 instructions and 355, 123 (2.89×) and 113 (3.14×) cycles.

An ideal 5-stage pipeline approaches a 5× speedup on long programs. Ours stays near 3× because the loops are short and end in a taken branch on almost every pass. [ADD: test results for Units 4 and 5]

13. Expected Benefits / Learning Outcomes

For pharmacy staff, the system keeps every medicine in one list, warns about low and near-expiry stock, and refuses a sale that would make the stock negative. Fewer stock-outs and fewer expired medicines on the shelf support SDG 3. For students, each value in the app can be traced to the hardware that produced it, so the system also works as a study aid for the course.

As a group we learnt to design an instruction set and write an assembler, to build an ALU and read its flags, to implement Booth's algorithm and both division methods, to do IEEE 754 arithmetic with correct rounding, and to measure pipeline speedup from real instruction and cycle counts. We also learnt to test a result automatically before trusting it, and to split the work across five members in one shared repository. These link to the course outcomes on processor organisation, the ALU and the control unit [ADD: CO numbers, if your faculty uses them] and on parallel architectures [ADD: once Units 4–5 are complete].

14. Challenges Faced

Non-restoring division gave wrong answers for large divisors because shifting A left could overflow it. We widened A to n + 2 bits (n bits plus carry and sign), and both methods then passed every test. Some floating-point results were rounded wrongly at first. We added guard, round and sticky bits with round-to-nearest-even and checked the output against the browser's float32 on 80,000 cases. Our first interface showed every Lab topic on one long page, which was dense and hard to follow, so we redesigned it as a simple medicine list in Stock and one Lab topic at a time. No faculty feedback on Report I had arrived before Report II, so we continued with the plan from Report I. [ADD: any teamwork, time or resource challenge in Month 3 and how the group handled it]

15. Future Improvements

[ADD: any part of Units 4–5 that is still unfinished] Beyond the current scope, the app could keep its data on a server so that several billing counters and branches share one live stock list, which is the shared-memory case from Units 4 and 5. It could also add separate logins for staff and managers, and run the scans on a larger dataset to see how the cycle counts grow.

16. Conclusion

We built a pharmacy inventory system in which each calculation runs on simulated hardware from the CAPP syllabus. Units 1 to 3 are complete. The ALU handles every sale and restock, IEEE 754 holds every price, and PharmaCPU runs the inventory

scans through a full instruction cycle under both hardwired and microprogrammed control [1]. With forwarding, the 5-stage pipeline runs our three programs 3.10× to 3.16× faster than the non-pipelined machine, and all 84,374 automated checks pass. [ADD: one sentence on the Unit 4–5 result] Building each unit for a real task taught us more than reading about it, because every design choice had to give a correct stock figure.

17. References

[1] M. M. Mano, Computer System Architecture, 3rd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1993.

[2] W. Stallings, Computer Organization and Architecture: Designing for Performance, 10th ed. Hoboken, NJ, USA: Pearson, 2016.

[3] C. Hamacher, Z. Vranesic and S. Zaky, Computer Organization, 5th ed. New York, NY, USA: McGraw-Hill, 2002.

[4] A. D. Booth, "A signed binary multiplication technique," The Quarterly Journal of Mechanics and Applied Mathematics, vol. 4, no. 2, pp. 236–240, 1951, doi: 10.1093/qjmam/4.2.236

[5] IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019, Jul. 2019, doi: 10.1109/IEEESTD.2019.8766229

[6] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," ACM Computing Surveys, vol. 23, no. 1, pp. 5–48, Mar. 1991, doi: 10.1145/103162.103163

[7] M. J. Flynn, "Very high-speed computing systems," Proceedings of the IEEE, vol. 54, no. 12, pp. 1901–1909, Dec. 1966, doi: 10.1109/PROC.1966.5273

[8] D. E. Culler, J. P. Singh and A. Gupta, Parallel Computer Architecture: A Hardware/Software Approach. San Francisco, CA, USA: Morgan Kaufmann, 1999.

18. Appendix (Source Code)

Repository: https://github.com/utk042/CAPP-PBL. Live app: https://utk042.github.io/CAPP-PBL/.

Excerpt 1. ALU with 3-bit function select and C, Z, N, V flags (Module 2), assets/js/alu.js, lines 37–68

```
function alu(op, a, b, n = 8) {
  const m = mask(n);
  a &= m;
  b &= m;
  const sign = 1 << (n - 1);
  let r = 0, c = 0, v = 0;
```

```
  switch (op) {
    case 'ADD': {
      const s = a + b;
      r = s & m;
      c = s > m ? 1 : 0;
      v = (~(a ^ b) & (a ^ r) & sign) ? 1 : 0;
      break;
    }
    case 'SUB': {
      // A - B implemented as A + (B' + 1): carry = 1 means "no borrow"
      const s = a + ((~b) & m) + 1;
      r = s & m;
      c = s > m ? 1 : 0;
      v = ((a ^ b) & (a ^ r) & sign) ? 1 : 0;
      break;
    }
    case 'AND': r = a & b; break;
    case 'OR': r = a | b; break;
    case 'XOR': r = a ^ b; break;
    case 'NOT': r = ~a & m; break;
    case 'SHL': r = (a << 1) & m; c = (a & sign) ? 1 : 0; break;
    case 'SHR': r = a >>> 1; c = a & 1; break;
    default: throw new Error('Unknown ALU op ' + op);
  }
  return { result: r, flags: { C: c, Z: r === 0 ? 1 : 0, N: (r & sign) ? 1 : 0, V: v } };
}
```

Excerpt 2. Booth’s signed multiplication (Module 2), assets/js/alu.js, lines 73–95

```
// Operands must lie in −(2^(n−1)−1) … 2^(n−1)−1 (A − M must not overflow n bits).
function booth(multiplicand, multiplier, n = 8) {
  const m = mask(n);
  const M = multiplicand & m;
  let A = 0, Q = multiplier & m, Q_1 = 0;
  const steps = [{ step: 0, action: 'Initialise', A: bin(A, n), Q: bin(Q, n), Q_1: Q_1 }];
  for (let i = 1; i <= n; i++) {
    const pair = `${Q & 1}${Q_1}`;
    let action = '';
    if (pair === '10') { A = (A - M) & m; action = 'A ← A − M'; }
    else if (pair === '01') { A = (A + M) & m; action = 'A ← A + M'; }
    else action = 'No operation';
    // arithmetic shift right of A, Q, Q-1
    Q_1 = Q & 1;
    Q = ((Q >>> 1) | ((A & 1) << (n - 1))) & m;
    A = ((A >>> 1) | (A & (1 << (n - 1)))) & m;
    steps.push({ step: i, pair, action: action + ', ASR', A: bin(A, n), Q: bin(Q, n), Q_1 });
  }
  // 2n-bit signed result
  const raw = A * 2 ** n + Q;
  const product = raw >= 2 ** (2 * n - 1) ? raw - 2 ** (2 * n) : raw;
  return { steps, product, bits: bin(A, n) + bin(Q, n) };
}
```

Excerpt 3. Restoring and non-restoring division (Module 2), assets/js/alu.js, lines 100–130

```
function divide(dividend, divisor, n = 8, method = 'restoring') {
  if (divisor === 0) throw new Error('Division by zero');
  const w = n + 2; // A register: n bits + carry + sign, so 2·A never overflows
  const m = mask(w);
  const qm = mask(n);
  const M = divisor & qm;
  let A = 0, Q = dividend & qm;
  const sgn = (x) => (x & (1 << (w - 1)) ? 1 : 0);
  const steps = [{ step: 0, action: 'Initialise', A: bin(A, w), Q: bin(Q, n) }];
  for (let i = 1; i <= n; i++) {
    // shift left A,Q
    A = ((A << 1) | (Q >>> (n - 1))) & m;
    Q = (Q << 1) & qm;
    let action;
    if (method === 'restoring') {
      A = (A - M) & m;
      if (sgn(A)) { Q &= ~1; A = (A + M) & m; action = 'SHL, A←A−M, A<0: Q0←0, restore A←A+M'; }
```

```
      else { Q |= 1; action = 'SHL, A←A−M, A≥0: Q0←1'; }
    } else {
      if (sgn(A)) { A = (A + M) & m; action = 'SHL, A<0: A←A+M'; }
      else { A = (A - M) & m; action = 'SHL, A≥0: A←A−M'; }
      if (sgn(A)) { Q &= ~1; action += ', Q0←0'; } else { Q |= 1; action += ', Q0←1'; }
    }
    steps.push({ step: i, action, A: bin(A, w), Q: bin(Q, n) });
  }
  if (method !== 'restoring' && sgn(A)) {
    A = (A + M) & m;
    steps.push({ step: '✓', action: 'Final: A<0 so A←A+M', A: bin(A, w), Q: bin(Q, n) });
  }
  return { steps, quotient: Q, remainder: A & qm };
}
```

Excerpt 4. IEEE 754 round-to-nearest-even with guard, round and sticky bits (Module 3), assets/js/alu.js, lines 210–221

```
// round a significand that carries `extra` bits below the 24-bit result (plus a sticky flag)
function roundNearestEven(sig, extra, sticky) {
  const one = 1n;
  const lsbMask = (one << BigInt(extra)) - one;
  const rem = sig & lsbMask;
  let q = sig >> BigInt(extra);
  const half = one << BigInt(extra - 1);
  const roundUp = rem > half || (rem === half && (sticky || (q & one) === one));
  const tie = rem === half && !sticky;
  if (roundUp) q += one;
  return { q, guard: rem >= half ? 1 : 0, rest: rem & (half - one) ? 1 : 0, sticky: sticky ? 1 :
0, roundUp, tie };
}
```

Excerpt 5. 5-stage pipeline timing with RAW stalls, forwarding and branch flush (Module 4), assets/js/cpu.js, lines 338–364

```
function pipeline(trace, { forwarding = true, limit = Infinity } = {}) {
  const rows = [];
  const list = trace.slice(0, limit);
  let prevStart = 0;
  list.forEach((t, i) => {
    const info = regsOf(t.dec);
    let start = i === 0 ? 1 : prevStart + 1;
    let reason = '';
    // data hazards with the previous two instructions
    for (let d = 1; d <= 2 && i - d >= 0; d++) {
      const p = rows[i - d];
      if (p.info.write === null || !info.reads.includes(p.info.write)) continue;
      const k = !forwarding ? 3 : p.info.load ? 2 : 1;
      if (p.start + k > start) { start = p.start + k; reason = `RAW on R${p.info.write} (from $
{p.mn})`; }
    }
    // control hazard: taken branch flushes the two wrongly fetched instructions
    if (i > 0 && rows[i - 1].taken && rows[i - 1].start + 3 > start) {
      start = rows[i - 1].start + 3; reason = 'Branch taken → flush 2';
    }
    rows.push({ i, mn: disasm(t.word), start, stalls: start - (i === 0 ? 1 : prevStart + 1),
reason, info, taken: t.taken && t.dec.fmt === 'B' });
    prevStart = start;
  });
  const n = rows.length;
  const total = n ? rows[n - 1].start + 4 : 0;
  const stalls = rows.reduce((s, r) => s + r.stalls, 0);
  return { rows, total, stalls, n, nonPipelined: n * 5, ideal: n ? n + 4 : 0, cpi: n ? total /
n : 0, speedup: total ? (n * 5) / total : 0 };
}
```

Faculty Evaluation:

To be completed by Faculty in Moodle after each review report submission, by conducting a proper viva-voce or presentation as per the department-level decision.